

我们的 CI/CD 实践经验

一、为什么需要 CI/CD

学生团队在项目开发中，通常会遇到这些问题：

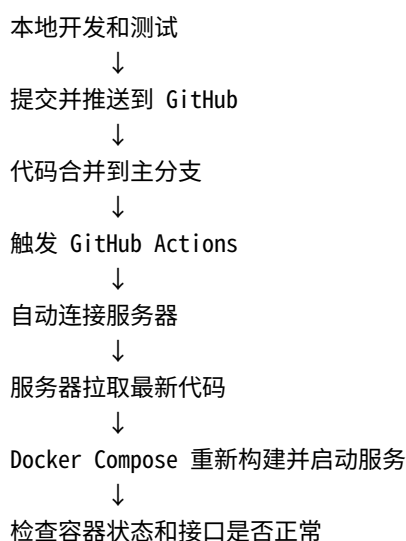
- 每次更新都要手动登录服务器部署；
- 不同成员的开发环境不一致；
- 容易上传错代码或漏掉配置；
- 不清楚服务器当前运行的是哪个版本；
- 项目一更新就可能出现前端、后端版本不一致；
- 部署工作长期集中在某一个人身上。

CI/CD 的作用，就是把代码测试、构建和部署流程尽量自动化。

开发人员只需要按照规范提交代码，系统就可以自动完成后续部署，减少重复操作和人为错误。

二、大概流程

我们项目采用的流程比较简单：



使用的主要工具包括：

- GitHub：管理代码和版本；
- GitHub Actions：自动执行部署；
- Docker：统一运行环境；
- Docker Compose：管理前端、后端和数据库等服务；
- Nginx：统一提供外部访问入口。

三、几点实践经验

1. 先手动部署成功，再做自动化

应先在服务器上手动完成一次完整部署，确认代码、配置、Docker、数据库和 Nginx 都可以正常运行，再将部署命令写入 GitHub Actions。

CI/CD 只是自动执行已经验证过的流程，不能代替部署方案设计。

2. 不要直接在服务器修改代码

代码应在本地完成修改，并统一提交到 GitHub。

服务器只负责拉取和运行代码，否则容易出现服务器版本和 GitHub 版本不一致，以及后续 `git pull` 冲突。

3. 配置和代码分开管理

数据库密码、服务器地址、密钥等敏感信息，不要直接提交到 GitHub。

可以分别放在：

- 服务器的 `.env` 文件；
- GitHub Secrets；
- Docker Compose 环境变量配置中。

4. 数据需要持久化

数据库、上传文件和运行结果要挂载到宿主机目录或 Docker Volume。

否则重新创建容器后，可能导致数据丢失。

5. 前端不要写死后端地址

前端访问后端时，尽量使用：

```
/api
```

然后由 Nginx 将请求转发到后端服务。

不要在代码中写死服务器 IP、端口或者 `localhost`，否则更换环境时会很麻烦。

6. 容器启动不代表系统正常

完成部署后，至少要检查：

```
docker compose ps
docker compose logs
```

最好再增加一个 `/health` 健康检查接口，用来判断后端、数据库等服务是否真正运行正常。

7. 自动部署流程不要设计得太复杂

对于一般学生项目，使用以下组合已经基本够用：

GitHub + GitHub Actions + SSH + Docker Compose + Nginx

先保证流程稳定可靠，再根据实际需要增加自动测试、测试环境、备份和回滚功能。

四、总结

CI/CD 的核心，不是使用多复杂的技术，而是把原来依赖个人手动操作的部署流程，变成一套固定、可重复、可追踪的流程。

最终达到的效果是：

开发人员提交代码
→ 系统自动部署
→ 团队检查运行结果
→ 出现问题可以根据版本和日志排查

这样可以减少重复部署工作，降低人为错误，也能避免整个团队只有一个人会部署项目。